

Assignment-3.3

2303A51303

Batch-05

Task01:

Prompt:

Generate a Python program that:

- Reads: Previous Units (PU), Current Units (CU), Type of Customer, Calculates units consumed, Implements logic directly in the main program (no functions)

Code:

```
'''Scenario
An electricity billing system must collect accurate consumer data.
Task Description
generate a Python program that:
• Reads:
o Previous Units (PU)
o Current Units (CU)
o Type of Customer
• Calculates units consumed
• Implements logic directly in the main program (no functions)'''
# Read Previous Units (PU)
PU = float(input("Enter Previous Units (PU): "))
# Read Current Units (CU)
CU = float(input("Enter Current Units (CU): "))
# Read Type of Customer
customer_type = input("Enter Type of Customer (Residential/Commercial): ").strip().lower()
# Calculate units consumed
units_consumed = CU - PU
# Initialize bill amount
bill_amount = 0.0
# Calculate bill based on customer type and units consumed
if customer_type == 'residential':
    if units_consumed <= 100:
        bill_amount = units_consumed * 0.5
    elif units_consumed <= 300:
        bill_amount = (100 * 0.5) + ((units_consumed - 100) * 0.75)
    else:
        bill_amount = (100 * 0.5) + (200 * 0.75) + ((units_consumed - 300) * 1.0)
elif customer_type == 'commercial':
    if units_consumed <= 100:
        bill_amount = units_consumed * 0.75
    elif units_consumed <= 300:
        bill_amount = (100 * 0.75) + ((units_consumed - 100) * 1.0)
    else:
        bill_amount = (100 * 0.75) + (200 * 1.0) + ((units_consumed - 300) * 1.5)
else:
```

```

        bill_amount = units_consumed * 0.75
    elif units_consumed <= 300:
        bill_amount = (100 * 0.75) + ((units_consumed - 100) * 1.0)
    else:
        bill_amount = (100 * 0.75) + (200 * 1.0) + ((units_consumed - 300) * 1.5)
else:
    print("Invalid customer type. Please enter 'Residential' or 'Commercial'.")
    exit()
# Display the bill amount
print(f"Total units consumed: {units_consumed}")
print(f"Total bill amount for {customer_type.capitalize()} customer: ${bill_amount:.2f}")
#Recursive implementation of fibonacci sequence
def fib(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)
n = 3
print(f"The {n}th Fibonacci number is: {fib(n)}")

```

Output:

```

he 3th Fibonacci number is: 2
S C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Users/pooji/AppData/Local/Programs/Python/Python313/python.exe c
/Users/pooji/OneDrive/Desktop/AI_Assistance_coding/Assign_3.3.py
Enter Previous Units (PU): 150
Enter Current Units (CU): 200
Enter Type of Customer (Residential/Commercial): Residential
Total units consumed: 50.0
Total bill amount for Residential customer: $25.00
he 3th Fibonacci number is: 2
S C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>

```

Explanation:

The program reads previous units, current units, and customer type to collect electricity consumption data. It calculates units consumed by subtracting previous units from current units.

Based on customer type (residential or commercial), different slab-wise billing rates are applied.

The total bill amount is computed and displayed along with units consumed.

A recursive Fibonacci function is included to demonstrate recursion by calculating the 3rd Fibonacci number.

Task02:

Prompt: Write a python code

Calculate Energy Charges (EC) and Use conditional statements based on: Domestic, Commercial, Industrial consumers

Improve readability using AI prompts such as: Simplify energy charge calculation logic, Optimize conditional statements

Code:

```
Assign_3.3.py > ...
20 def calculate_energy_charges(units_consumed, customer_type):
21     if customer_type == 'Domestic':
22         if units_consumed <= 100:
23             ec = units_consumed * 0.5
24         elif units_consumed <= 300:
25             ec = (100 * 0.5) + ((units_consumed - 100) * 0.75)
26         else:
27             ec = (100 * 0.5) + (200 * 0.75) + ((units_consumed - 300) * 1.0)
28     elif customer_type == 'Commercial':
29         if units_consumed <= 200:
30             ec = units_consumed * 0.75
31         elif units_consumed <= 500:
32             ec = (200 * 0.75) + ((units_consumed - 200) * 1.0)
33         else:
34             ec = (200 * 0.75) + (300 * 1.0) + ((units_consumed - 500) * 1.25)
35     elif customer_type == 'Industrial':
36         if units_consumed <= 500:
37             ec = units_consumed * 1.0
38         else:
39             ec = (500 * 1.0) + ((units_consumed - 500) * 1.5)
40     else:
41         raise ValueError("Invalid customer type")
42
43     return ec
44 # Improved Code
45 def calculate_energy_charges_optimized(units_consumed, customer_type):
46     rates = {
47         'Domestic': [(100, 0.5), (200, 0.75), (float('inf'), 1.0)],
48         'Commercial': [(200, 0.75), (300, 1.0), (float('inf'), 1.25)],
49         'Industrial': [(500, 1.0), (float('inf'), 1.5)]
50     }
51
```

```

* Assign_3.3.py > ...
43     return ec
44 # Improved Code
45 def calculate_energy_charges_optimized(units_consumed, customer_type):
46     rates = {
47         'Domestic': [(100, 0.5), (200, 0.75), (float('inf'), 1.0)],
48         'Commercial': [(200, 0.75), (300, 1.0), (float('inf'), 1.25)],
49         'Industrial': [(500, 1.0), (float('inf'), 1.5)]
50     }
51
52     if customer_type not in rates:
53         raise ValueError("Invalid customer type")
54
55     ec = 0
56     remaining_units = units_consumed
57
58     for limit, rate in rates[customer_type]:
59         if remaining_units <= 0:
60             break
61         units_at_rate = min(remaining_units, limit)
62         ec += units_at_rate * rate
63         remaining_units -= units_at_rate
64
65     return ec
66 # Sample Execution
67 if __name__ == "__main__":
68     units= 450
69     customer= 'Commercial'
70     original_ec = calculate_energy_charges(units, customer)
71     optimized_ec = calculate_energy_charges_optimized(units, customer)
72     print(f"Original EC for {customer} with {units} units: {original_ec}")
73     print(f"Optimized EC for {customer} with {units} units: {optimized_ec}")

```

Output:

```

PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
./Users/pooji/OneDrive/Desktop/AI_Assistance_coding/Assign_3.3.py
Original EC for Commercial with 450 units: 400.0
Optimized EC for Commercial with 450 units: 400.0
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>

```

Explanation:

The original code uses nested if-elif conditions, making it longer and harder to maintain.

The optimized code stores slab limits and rates in a dictionary, improving readability and structure.

Adding or modifying tariff slabs is easier in the optimized version without changing logic flow. The optimized approach reduces code repetition by using a loop for calculations. Both versions produce the same correct energy charge, but the optimized code is cleaner and scalable.

Task03:

Prompt: Python program that:

Uses user-defined functions to Calculate Energy Charges, Calculate Fixed Charges,
Returns calculated values

Code:

```
# Function to calculate Energy Charges
def calculate_energy_charges(units_consumed, rate_per_unit):
    energy_charges = units_consumed * rate_per_unit
    return energy_charges

# Function to calculate Fixed Charges
def calculate_fixed_charges(fixed_charge_rate):
    return fixed_charge_rate

# Test cases to validate the functions
if __name__ == "__main__":
    # Test case 1
    units = 150
    rate = 0.12
    fixed_charge = 50.0
    ec = calculate_energy_charges(units, rate)
    fc = calculate_fixed_charges(fixed_charge)
    print(f"Test Case 1 - Energy Charges: ${ec:.2f}, Fixed Charges: ${fc:.2f}")

    # Test case 2
    units = 200
    rate = 0.15
    fixed_charge = 75.0
    ec = calculate_energy_charges(units, rate)
    fc = calculate_fixed_charges(fixed_charge)
    print(f"Test Case 2 - Energy Charges: ${ec:.2f}, Fixed Charges: ${fc:.2f}")
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Users/pooji/AppData/Local/Programs/Python/Python313/py
:/Users/pooji/OneDrive/Desktop/AI_Assistance_coding/Assign_3.3.py
Test Case 1 - Energy Charges: $18.00, Fixed Charges: $50.00
Test Case 2 - Energy Charges: $30.00, Fixed Charges: $75.00
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

This program demonstrates reusable billing logic using user-defined functions.

The `calculate_energy_charges()` function computes energy cost by multiplying units consumed with the rate per unit and returns the value.

The `calculate_fixed_charges()` function returns a fixed charge amount applicable to the

consumer. In the main block, multiple test cases are executed to validate the functions with different inputs. This approach improves code reusability, clarity, and makes billing calculations easy to maintain.

Task04:

Prompt: Write a python code to calculate the FC ,CC,ED, add electricity duty calculation

Code:

```
Assign_3.3.py > ...
15 def calculate_electricity_bill(units_consumed):
16     # Constants
17     RATE_PER_UNIT = 5.0 # Cost per unit of electricity
18     FIXED_CHARGES = 50.0 # Fixed charges
19     CUSTOMER_CHARGES = 20.0 # Customer charges
20     ELECTRICITY_DUTY_RATE = 0.05 # 5% Electricity Duty
21
22     # Calculate Energy Charges (EC)
23     energy_charges = units_consumed * RATE_PER_UNIT
24
25     # Calculate Electricity Duty (ED)
26     electricity_duty = energy_charges * ELECTRICITY_DUTY_RATE
27
28     # Total Bill Calculation
29     total_bill = energy_charges + FIXED_CHARGES + CUSTOMER_CHARGES + electricity_duty
30
31     # Print individual charge values
32     print(f"Energy Charges (EC): ${energy_charges:.2f}")
33     print(f"Fixed Charges (FC): ${FIXED_CHARGES:.2f}")
34     print(f"Customer Charges (CC): ${CUSTOMER_CHARGES:.2f}")
35     print(f"Electricity Duty (ED): ${electricity_duty:.2f}")
36     print(f"Total Bill: ${total_bill:.2f}")
37     return total_bill
38
39 # Example usage
40 units = 150 # Example units consumed
41 calculate_electricity_bill(units)
```

Output:

```
Test Case 2 - Energy Charges: $750.00, Fixed Charges: $50.00
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Users/pooji/AppData/Local/Programs/Python/Python313/python.exe c
:/Users/pooji/OneDrive/Desktop/AI_Assistance_coding/Assign_3.3.py
Energy Charges (EC): $750.00
Fixed Charges (FC): $50.00
Customer Charges (CC): $20.00
Electricity Duty (ED): $37.50
Total Bill: $857.50
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```


Explanation: This program calculates a complete electricity bill by including multiple charges. It first computes Energy Charges (EC) based on units consumed and a fixed per-unit rate.

Then it calculates Electricity Duty (ED) as a percentage of the energy charges.

Fixed Charges (FC) and Customer Charges (CC) are added as constant values.

Finally, all charges are summed and displayed clearly to produce the total bill.

Task05:

Prompt: Develop the final Python application to:

Calculate total bill: Total Bill = EC + FC + CC + ED

Display Energy Charges (EC), Fixed Charges (FC), Customer Charges (CC), Electricity Duty (ED), Total Bill Amount

Code:

```
def calculate_electricity_bill(units_consumed):
    # Define the rates and charges
    energy_charge_rate = 5.0 # per unit
    fixed_charge = 50.0 # fixed charge
    customer_charge = 20.0 # customer charge
    electricity_duty_rate = 0.05 # 5% of energy charges

    # Calculate Energy Charges (EC)
    energy_charges = units_consumed * energy_charge_rate

    # Calculate Electricity Duty (ED)
    electricity_duty = energy_charges * electricity_duty_rate

    # Calculate Total Bill
    total_bill = energy_charges + fixed_charge + customer_charge + electricity_duty

    # Display the breakdown of charges
    print(f"Energy Charges (EC): ${energy_charges:.2f}")
    print(f"Fixed Charges (FC): ${fixed_charge:.2f}")
    print(f"Customer Charges (CC): ${customer_charge:.2f}")
    print(f"Electricity Duty (ED): ${electricity_duty:.2f}")
    print(f"Total Bill Amount: ${total_bill:.2f}")

    return total_bill

# Main function to run the program
def main():
    try:
        # Input: Units consumed
        units_consumed = float(input("Enter the number of units consumed: "))
        if units_consumed < 0:
            raise ValueError("Units consumed cannot be negative.")

        # Calculate and display the electricity bill
        calculate_electricity_bill(units_consumed)

    except ValueError as e:
        print(f"Invalid input: {e}")
```

```

def calculate_electricity_bill(units_consumed):
    # Calculate Energy Charges (EC)
    energy_charges = units_consumed * 20
    # Calculate Fixed Charges (FC)
    fixed_charges = 50
    # Calculate Customer Charges (CC)
    customer_charges = 20
    # Calculate Electricity Duty (ED)
    electricity_duty = units_consumed * 1
    # Calculate Total Bill Amount
    total_bill = energy_charges + fixed_charges + customer_charges + electricity_duty

    print(f"Electricity Duty (ED): ${electricity_duty:.2f}")
    print(f"Total Bill Amount: ${total_bill:.2f}")

    return total_bill

# Main function to run the program
def main():
    try:
        # Input: Units consumed
        units_consumed = float(input("Enter the number of units consumed: "))
        if units_consumed < 0:
            raise ValueError("Units consumed cannot be negative.")

        # Calculate and display the electricity bill
        calculate_electricity_bill(units_consumed)

    except ValueError as e:
        print(f"Invalid input: {e}")

# Run the main function
if __name__ == "__main__":
    main()

```

Output:

```

Total Bill Amount: $91.00
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Users/pooji/AppData/Local/Programs/Python/Python313/python.exe c
:/Users/pooji/OneDrive/Desktop/AI_Assistance_coding/Assign_3.3.py
Enter the number of units consumed: 4
Energy Charges (EC): $20.00
Fixed Charges (FC): $50.00
Customer Charges (CC): $20.00
Electricity Duty (ED): $1.00
Total Bill Amount: $91.00
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>

```

Explanation:

This program accurately calculates the total electricity bill by summing Energy Charges, Fixed Charges, Customer Charges, and Electricity Duty.

Clear variable names and step-by-step calculations make the code easy to read and understand.

Input validation ensures realistic values by preventing negative unit entries.

The detailed output displays each charge separately, improving transparency for users.